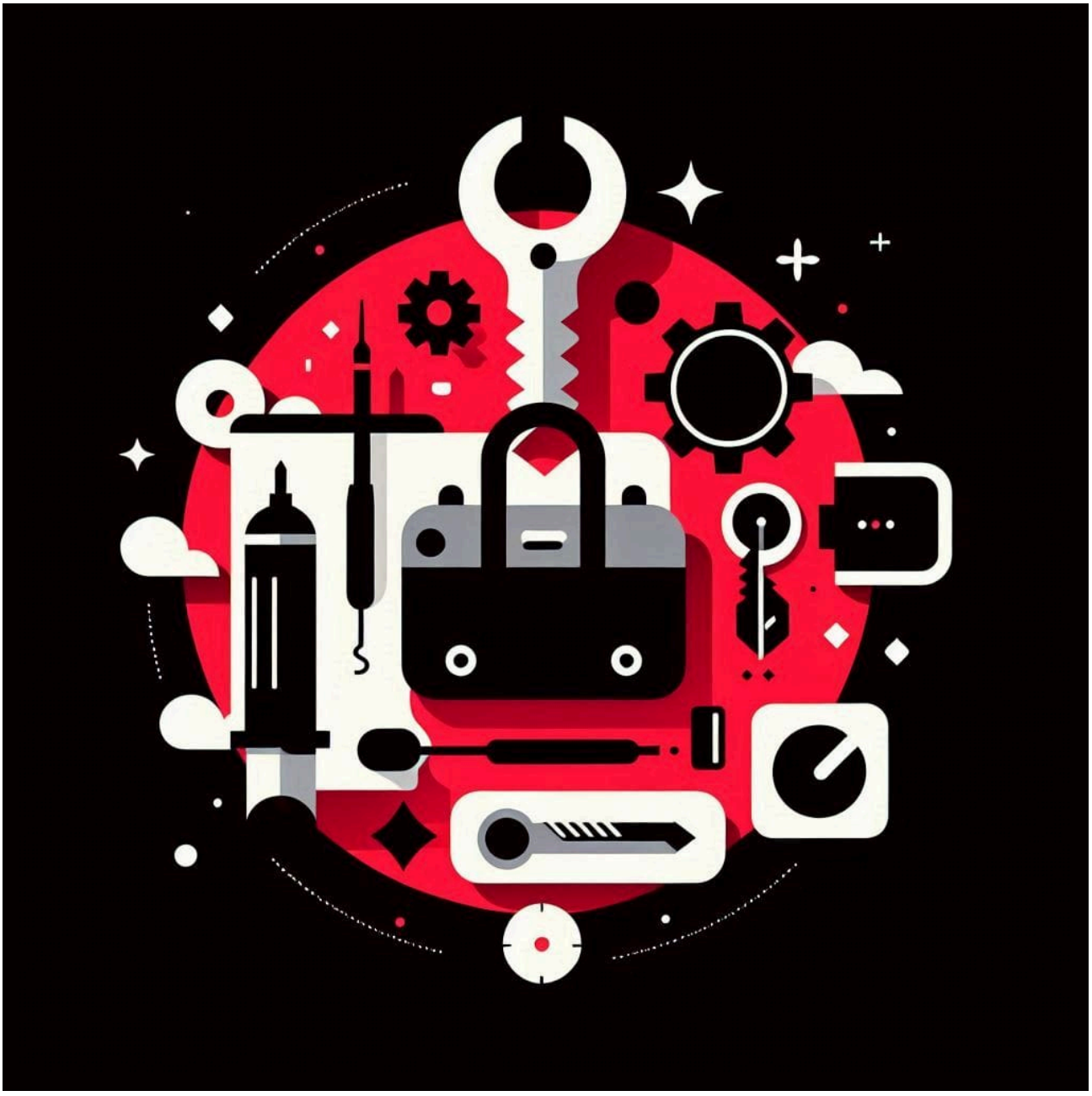
Use Pwntools to level up your Toolset

minder-security.ghost.io/elevate-your-skills-using-python-pwntools

Richard Minder April 29, 2024



While the standard hacking toolset can get you far, there will be some challenges that require you to be able to craft your own. I have used pwntools numerous times throughout my learning path, and now I want to show you why you should add this powerful tool to your arsenal.

Why you should use Pwntools#

While there are vulnerabilities which can be easily exploited using regular Python scripting or even commonly used tools, there can be scenarios where you want to **keep your**

coding efforts to a minimum and save time.

These scenarios might include Analysis, Patching and Exploitation of processes/binaries, network connections, serial connections or SSH connections.

Use case example: You need to successfully exploit a running process on another endpoint. You could now go ahead and write a script using vanilla Python, where the socket connection, the processing of received data, and the creation/modification of shellcode would all require you to write your own code. The coding and debugging of that code would consume a lot of valuable time, but luckily there is pwntools, which provides you with libraries written specifically for such tasks, and much more.

The idea behind pwntools#

The philosophy behind a pwntools script is quite simple actually:

- Attach me to a local process, executable file, socket, SSH connection or serial port
- Let me read and analyze
- Let me write and modify
- Let me help you automate debugging with GDB
- Let me easily work with data
- Let me easily convert your clear-text assembly instructions to byte code and vice versa

If you are interested in **local processes or executable files**:

- If it's a **local process**, let me:
  - Read/write stdin/stdout/stderr
  - Read/write its virtual memory
  - Debug using GDB
- If it's an **executable file**, let me read the binary code inside of it and work with it:
  - Spawn a process from it
  - Analyze Symbols, Instructions, Data, Addresses and more
  - Assemble/Disassemble instructions
  - Modify the instructions inside the file (binary patching)
  - Identify ROP-Gadgets

The **functionalities regarding data** come in very handy:

Let me easily work with data and datatypes:

- Packing and unpacking
- Hashing and encoding
- Generate patterns (mostly for offset identification)
- Generate bytes from assembly instructions on-the-go
- Disassemble existing bytes to clear-text, readable assembly instructions

Step 1 - Import and configure Context

First, let's import everything inside Pwntools:

```
from pwn import *
```

After that, we can set some runtime variables to further define what context we are working in. We can do so by either setting the variable ourselves, or by using *context.update()*:

```
context.binary = "/home/user/lab/binary"

context.os = "linux"

context.arch = "amd64"
context.bits = 64
context.endian = "little"

context.update(arch="i386")
```

Step 2 - Create a Tube

Tubes are connections established with:

- Local Processes
- TCP or UDP sockets
- Remote SSH connections
- Serial Port I/O

You can then proceed to read or write to the defined tube and perform actions on it. A quick example:

```
io = process("/home/user/lab/test")
io = process(argv=["foo", "bar"], executable="/home/user/lab/test")


io = remote("127.0.0.1", "1234")


io = remote.fromsocket(s)
```

```
io = ssh("username", "127.0.0.1", password="password123")


io = serialtube('dev/ttyUSB0', baudrate=115200)
```

For more information, see:

[pwnlib.tubes — Talking to the World! — pwntools 4.12.0 documentation]

Step 3 - Read/Write to a Tube

After creating tubes, you can easily perform read or write actions on them:

```
io.recv(n)
io.recvline()
io.recvuntil(b'?')
io.recvrepeat(5)
io.clean()


io.send(data)
io.sendline(data)
io.sendlineafter(b'Write 0xdeadbeef as bytes:', p64(0xefbeadde))
```

You can also initiate an interactive session:

```
io.interactive()
```

Binary Patching

Binary Patching refers to the static modification of executable files. In simple terms: you change the code inside an executable file.

Let's say you downloaded a trial version of a photo editing software, which requires you to buy a license after 30 days. You could go ahead and buy said license, or you could just modify the application's code and overwrite the license check with your own instructions, like a NOP sled for example.

Let's take a look at how Pwntools can aid us in this.

Step 1 - Defining the ELF

First, we need to define which ELF we want to work with. An Example:

```
e = ELF("/home/user/lab/myprogram")
```

Once the ELF has been specified, we can analyze it. We can also get quick access to addresses of the Symbols, Global Offset Table (GOT), Process Linkage Table (PLT):

```
e.symbols
e.got
e.plt
```

Step 2 - Reading from the ELF

We can start with listing the symbols defined in the ELF:

```
for key, address in e.symbols.iteritems():
  print(f"{hex(address)}: {key}")
```

If you are only interested in the address of the main function, you could print it like so:

```
print(f"{hex(e.sym.main)}: main")
```

Once we printed the symbols and their corresponding addresses, we can analyze the binary in our reversing tool of choice.

What is also possible is to search for a specific byte sequence inside the ELF. In this snippet, I'm looking for all addresses where the string "/bin/sh" occurs:

```
for addr in e.search("/bin/sh\x00"):
  print(f"{addr}: /bin/sh")
```

Step 3 - Patching the ELF

After we identified the addresses of the instructions we want to patch, we can proceed in doing so.

In this example, I'm overwriting a call to a function `check`, which was checking if I passed a challenge or not with `ret`, practically skipping the checking part of the crackme:

```
e.asm(e.symbols['check'], 'ret')
```

This is a very simple example, but you could also overwrite a `jmp` instruction with `nop` as follows:

```
nop = asm('nop')

e.write(0x0804840a, nop)
```

Step 4 - Running the ELF

After we patched it, we can now run it. You would probably do this using your terminal, but if you want to work with the newly spawned process, Pwntools provides you with the necessary tools to do so seamlessly:

```
io = e.process()
```

```
io = process(e.path)
```

You also have the possibility to launch the process with GDB attached to it:

```
io = e.debug()
```

```
io = gdb.debug(e.path)
```